FarmerApp — Product, Design & Engineering Specification

2026-09-17 · @Someone

0. How to use this document

This is the single source of truth for FarmerApp: requirements, design brief, infrastructure, delivery process and the build loop. Work through it in this order.

1. **Confirm decisions.** Read sections 1–8 and resolve the open questions in 0.3 before any code.
2. **Design.** Paste the prompt in section 13 into Claude Design. Iterate until every screen in the screen inventory exists in both themes. Export the design tokens and screens.
3. **Seed the repo.** Commit this document as `docs/SPEC.md`, the design export under `design/`, and the `CLAUDE.md` from section 14.2.
4. **Build in loops.** Run Claude Code milestone by milestone (section 14). Each loop iteration picks the next unchecked requirement, implements it with tests, and stops only when the milestone's definition of done passes in CI.
5. **Release.** Promote through dev → staging → prod using the pipeline in section 10.

0.1 Requirement IDs

Every requirement has a stable ID. Code, tests, PR titles and commits reference it (for example `feat(FA-012): farm geofence capture`).

Prefix	Area
FA	Field agent app
FM	Farmer app
ST	Store mode (billing, stock)
WC	Web console
DX	Diagnosis engine
VV	Visit verification and location

Prefix	Area
AN	Analytics layer
VA	Validation agents
NF	Non-functional
SEC	Security
OPS	Monitoring and operations

Priority: **P0** = pilot (Rabi 2026), **P1** = full rollout (Kharif 2027), **P2** = SaaS (FY28).

0.2 Locked decisions

Decision	Choice
Mobile	Flutter 3.x, two apps from one monorepo, Android first (min SDK 26), iOS later for farmer app only
Web console	Next.js (App Router) + TypeScript, deployed to Cloud Run
API	Python 3.12 + FastAPI on Cloud Run, OpenAPI-first
Database	Cloud SQL for PostgreSQL 16 with PostGIS and pgvector
Region	asia-south1 (Mumbai), DR backups to asia-south2 (Delhi)
IaC	Terraform, one module set, three environments
CI/CD	GitHub Actions with Workload Identity Federation (no JSON keys)
Auth	Phone + OTP (own service, SMS via DLT-registered provider, WhatsApp fallback), JWT access + refresh
Payments	Razorpay (UPI dynamic QR, payment links, webhooks); Cashfree as swappable adapter
Tenancy	Shared schema, `tenant_id` on every row, Postgres row-level security

Decision	Choice
Analytics	Datastream CDC → BigQuery, dbt models, Looker Studio for owners + in-console dashboards

0.3 Open questions (resolve before M1)

- [ ] Languages for launch: Telugu, Kannada, Hindi, Marathi, English — confirm which states the first 5 pilot stores are in.
- [ ] Top 6 crops for the diagnosis corpus v1 (proposal: paddy, cotton, chilli, maize, tomato, groundnut).
- [ ] Agronomist: named person, hours per week, and approval authority for recommendations.
- [ ] Travel allowance rate per km and incentive rules, so visit data can drive payouts.
- [ ] Razorpay vs existing bank/POS provider already used in stores.
- [ ] Does FCC already use accounting software (Tally, Zoho Books) that store billing must sync with?
- [ ] Product master: who owns SKU data, and where are CIB&RC label claims recorded today?
- [ ] Legal review of the employee location policy and farmer consent text (DPDP obligations apply from 13 May 2027).

1. Product overview

FarmerApp is the operating system for a regional agri-input retail chain: it connects field officers, stores and farmers, and turns every farm visit and crop photo into verified data and own-brand sales. It launches inside Future Crop Care (FCC), then sells to similar chains as multi-tenant SaaS.

1.1 Business context (from the FCC FY26–FY30 model)

- Stores grow from 24 (FY26) to 50 (FY30); each store has 1 manager, 1 accountant and 1 field officer; 1 cluster head per 8 stores; 1 store auditor per 15 stores.
- Own-brand margins are 30–50% versus 2–15% for multi-brand products. Own-brand share of sales is planned to rise from about 33% to 46%.
- B2B sales start in FY27 (₹2 Cr) and reach ₹8 Cr by FY30.
- Farmers commonly buy on credit until harvest, so receivables and collections matter.

1.2 Surfaces

Surface	Users	Distribution
Field agent app ("Mitra")	Field officers, cluster heads	Android, private/managed Play distribution
Farmer app ("Kisan")	Farmers	Public Play Store; iOS in P2
Store mode	Store managers, accountants	Inside the field agent app, role-gated; works on a tablet
Web console	Owner, directors, cluster heads, auditors, agronomist, tenant admins	Browser

Working names "Mitra" and "Kisan" are placeholders; brand names are tenant-configurable.

1.3 Personas

Persona	Goal	Pain today
Owner / director	See which stores, officers and farmers drive profit	No reliable view of field work; data arrives late on WhatsApp and paper
Cluster head	Coach 8 stores, fix weak officers early	Cannot verify visits; relies on self-reporting
Field officer	Build farmer trust, hit targets, get travel and incentive paid	Paperwork, disputed claims, no product answers in the field
Store manager	Sell, collect credit, keep stock	Manual khata, stockouts in peak season
Agronomist	Give correct advice at scale	Same questions repeated; no photo history
Farmer	Save the crop, buy the right product at a fair price, on credit if needed	Uncertain diagnosis, fake products, travel to shop

1.4 Glossary

Term	Meaning
Tenant	A company using the platform (FCC is tenant 1)
Beat	A planned route of villages for a field officer on a day
Visit	A verified field-officer interaction with a farmer at a farm or village point
Proof-of-visit (PoV)	Geofence + timestamp + photo + farmer confirmation bundle that verifies a visit
Khata	Farmer credit ledger
Label claim	Crop–pest combination a pesticide is registered for (CIB&RC)
Corpus	Agronomist-verified set of crop images, problems, and treatments used for diagnosis
Own brand	Tenant's white-label products; carry higher margin

1.5 Success metrics (first 12 months at FCC)

Metric	Target
Farmers registered per field officer	400+ by end of Kharif 2027
Verified visits per officer per working day	6+
Share of visits with farmer confirmation	60%+
Farmer app monthly active users	30% of registered farmers
Diagnosis answered from corpus without escalation	80%+ by month 9
Own-brand share of store sales	+5 percentage points vs baseline
Credit collected on time (by due date)	85%+
Officer weekly retention	No officer attrition attributed to the app

Targets are proposals; confirm with FCC management before M1.

2. Functional requirements

Each requirement is testable; the acceptance criteria column is what the automated test or QA script checks.

2.1 Common to all surfaces

ID	Requirement	Pri	Acceptance criteria
FA-001	Phone + OTP login; 6-digit OTP, 5-minute expiry, 3 resends per 15 min, lockout after 5 wrong attempts	P0	OTP arrives by SMS within 20 s on test SIMs; WhatsApp fallback after 30 s; lockout verified
FA-002	Role-based access: owner, director, cluster_head, auditor, store_manager, accountant, field_officer, agronomist, tenant_admin, farmer	P0	Every API route has a role policy; forbidden calls return 403 and are logged

ID	Requirement	Pri	Acceptance criteria
FA-003	Tenant isolation on every record	P0	RLS test suite proves tenant A cannot read or write tenant B data via API or SQL
FA-004	Localisation: English, Telugu, Kannada, Hindi, Marathi at launch; all strings externalised (ARB files)	P0	No hard-coded UI strings (lint check); language switch without restart
FA-005	Voice input for free-text fields (Android speech-to-text, regional language)	P1	Note field accepts dictated Telugu/Hindi text
FA-006	Audit trail: every create/update/delete stores actor, time, device, before/after	P0	Audit row exists for each mutation in integration tests
FA-007	Push notifications via FCM with in-app inbox	P0	Notification delivered and listed in inbox
FA-008	Tenant branding: logo, app name, primary colour, support phone	P2	Second test tenant renders with its brand without a code change

2.2 Field agent app

ID	Requirement	Pri	Acceptance criteria
FA-010	Duty on/off toggle; location logging runs only while on duty, with Android foreground service and persistent notification	P0	No location points recorded while off duty; notification visible while on
FA-011	First-run disclosure screen explaining what is collected, when, why (TA and visit credit), who sees it; explicit acceptance stored with policy version	P0	Acceptance record with version and timestamp; app blocks duty until accepted

ID	Requirement	Pri	Acceptance criteria
FA-012	Farmer registration: name, phone (OTP-verified), village, landmark, farm boundary (walk-around or 4-point tap), area in acres, crops with variety and sowing date, irrigation type, soil type, land ownership (own/leased), preferred language, consent for messages	P0	Registration saved offline and synced; duplicate phone blocked; consent record created
FA-013	Multiple farms per farmer, multiple crops per farm per season	P0	Farmer with 2 farms and 3 crop plots can be created
FA-014	Visit check-in: allowed only within farm geofence (default 150 m, tenant-configurable) or at a registered village meeting point	P0	Check-in outside geofence is blocked with reason; override requires note + cluster head approval
FA-015	Visit capture: purpose (advisory, scouting, demo, collection, sale, meeting), notes, at least one geotagged photo taken in-app (no gallery), crop stage	P0	Gallery upload disabled for visit photos; EXIF and server time stored
FA-016	Farmer confirmation: OTP to farmer phone, or push prompt in farmer app, or QR shown on farmer app	P0	Visit marked confirmed only after one channel succeeds
FA-017	Visit check-out with auto duration; minimum duration configurable (default 5 min)	P0	Visits under minimum flagged, not rejected
FA-018	Beat plan: daily list of villages and farmers suggested by the system (due follow-ups, pest alerts, credit due) and editable by the officer	P1	Plan generated at 6:00 local; officer can reorder and add
FA-019	Scouting: photo diagnosis using the diagnosis engine (DX), attach result to farmer and crop plot	P0	Result stored with confidence and model version

ID	Requirement	Pri	Acceptance criteria
FA-020	Order capture for a farmer: product search, own-brand badge, stock at nearest store, quantity, price, delivery or pickup	P0	Order reaches store queue; stock reserved
FA-021	Payment collection: UPI dynamic QR, payment link via SMS/WhatsApp, cash with receipt photo, or credit (khata)	P0	Razorpay webhook marks paid; cash requires store reconciliation
FA-022	Khata view per farmer: balance, due dates linked to harvest month, payment history	P0	Balance equals sum of ledger entries
FA-023	Travel allowance: daily km computed from on-duty track (snapped, capped), expense claims with receipt photo	P1	Km within 10% of Google Maps route distance on test routes
FA-024	My performance: visits, farmers added, confirmations, sales, collections, incentive earned to date	P1	Numbers match console for the same officer and period
FA-025	Offline-first: all capture works offline for 3 days; sync queue with conflict resolution (server wins for money, last-writer for notes)	P0	Airplane-mode test for 72 h then sync with zero data loss
FA-026	Demo plots: register demo, track stages with photos, compare to farmer practice	P1	Demo lifecycle from creation to yield entry
FA-027	Farmer meeting: group session with attendance by farmer OTP or QR scan	P1	Attendance list linked to farmers
FA-028	Mock-location, rooted device and time-tamper detection; flagged to validation agents, not blocked	P0	Mock location app on test device raises flag

2.3 Store mode

ID	Requirement	Pri	Acceptance criteria
ST-001	Point of sale: search or scan barcode, cart, farmer lookup by phone, GST invoice (HSN, CGST/SGST/IGST)	P0	Invoice PDF matches GST format; shareable on WhatsApp
ST-002	Payment modes: UPI QR, card via existing terminal (manual reference), cash, credit	P0	Day-end totals by mode
ST-003	Stock: goods receipt, transfers between stores, batch and expiry tracking, damage write-off	P0	Stock per SKU per batch correct after each movement
ST-004	Low-stock and expiry alerts (60/30 days)	P1	Alerts generated nightly
ST-005	Order queue from field officers and farmer app; pack, handover, delivery status	P0	Status visible to farmer and officer
ST-006	Day close: cash count, variance, deposit slip photo	P0	Variance over threshold flags auditor
ST-007	Credit approval: limit per farmer set by store manager; above limit needs cluster head	P0	Sale beyond limit blocked without approval
ST-008	Export to Tally/Zoho (CSV or API)	P1	Daily export file validated by accountant

2.4 Farmer app

ID	Requirement	Pri	Acceptance criteria
FM-001	Login with phone + OTP; language chosen on first screen with audio prompt	P0	Login in under 60 s on a low-end phone

ID	Requirement	Pri	Acceptance criteria
FM-002	Consent screen: plain-language notice (in chosen language) for data use and marketing, separately toggleable	P0	Marketing consent stored separately; withdrawable in settings
FM-003	Profile: farms and crops, pre-filled if registered by an officer; farmer can confirm or edit	P0	Officer-registered farmer sees own data after login
FM-004	Crop doctor: guided camera (frame the affected leaf, tips on light/focus), crop auto-detect with manual override, up to 3 photos	P0	Blurry photo rejected with retake tip
FM-005	Diagnosis result: problem name, confidence band (high/medium/low shown as words), symptoms to confirm, organic and chemical options, dosage per acre and per tank, safety interval, products at nearest store with price and stock	P0	Low-confidence shows "provisional — expert will confirm" and notifies on update
FM-006	One-tap actions from result: order for pickup/delivery, call field officer, ask expert	P0	Order reaches store queue
FM-007	Crop calendar by crop and sowing date: tasks (sow, irrigate, fertilise, spray, harvest) with reminders	P1	Calendar regenerates when sowing date changes
FM-008	Weather: 7-day forecast and spray window advice at farm location	P1	Spray advice hides windows with rain or wind above threshold
FM-009	Pest and disease alerts by district, based on diagnosis clusters	P1	Alert sent when cluster rule fires
FM-010	My orders, khata balance, pay dues via UPI	P0	Payment updates balance within 1 min of webhook
FM-011	Visit confirmation prompt and rating (1–5 + reason)	P0	Rating stored against visit

ID	Requirement	Pri	Acceptance criteria
FM-012	Mandi prices for the farmer's crops (public data source)	P2	Prices shown with date and source
FM-013	Referral: invite a farmer, credited to referrer and nearest store	P2	Referral attributed
FM-014	App size under 25 MB; works on Android 8+, 2 GB RAM	P0	APK size checked in CI
FM-015	Delete account and data request from settings	P0	Request creates ticket; data removed within policy timeline

2.5 Web console

ID	Requirement	Pri	Acceptance criteria
WC-001	Owner home: KPIs, trends, map, alerts (see section 5)	P0	Loads in under 3 s for 50 stores
WC-002	Field team: list, profile, live duty status, day timeline, visit map replay	P0	Replay shows track, visits and gaps
WC-003	Farmers: searchable directory, profile with farms, crops, visits, diagnoses, purchases, khata, consent status	P0	Profile reflects latest sync
WC-004	Stores: sales, stock, credit book, day-close variance	P0	Store P&L view by month
WC-005	Product master: SKUs, own-brand flag, label claims (crop × pest × dose), price lists by store	P0	Recommendation cannot use a SKU without a matching label claim
WC-006	Agronomist review queue: low-confidence and flagged diagnoses, correct label, add to corpus	P0	Correction updates farmer result and notifies farmer
WC-007	Corpus manager: problems, images, treatments, versions, approvals	P0	Only approved entries used in production

ID	Requirement	Pri	Acceptance criteria
WC-008	Campaigns: build segment (crop, district, stage, purchase history, consent), send SMS/WhatsApp/push, track conversion	P1	Only consented farmers are selectable
WC-009	Targets and incentives: set monthly targets per officer and store; incentive rules engine	P1	Incentive computed matches rule on test data
WC-010	Validation agent inbox (section 6): flags, evidence, resolve/escalate	P0	Each flag has an owner and an SLA
WC-011	User and role management, device binding, forced logout	P0	Revoked user loses access within 5 min
WC-012	Settings: geofence radius, minimum visit time, TA rate, credit limits, languages, branding	P0	Changes audited
WC-013	Exports: CSV/XLSX of any table view; scheduled email reports	P1	Weekly report delivered Monday 8:00 IST
WC-014	Tenant admin (platform level): create tenant, plan, limits, billing	P2	New tenant usable in under 30 min

2.6 Permissions matrix

Data	Owner / director	Cluster head	Auditor	Store manager	Field officer	Agronomist	Farmer
All stores KPIs	Read	Own cluster	Read	Own store	—	—	—
Officer location track	Read	Own cluster	Read	—	Own	—	—
Farmer profile	Read	Own cluster	Read	Own store	Own farmers	Read (crop data)	Own
Khata	Read	Own cluster	Read	Read/write own store	Read own farmers	—	Own
Corpus	Read	—	—	—	—	Read/write	—
Settings	Write	—	—	—	—	—	—

Location tracks are never shown to store managers or other officers. Tracks older than 180 days are aggregated and raw points deleted.

3. Diagnosis engine

Diagnosis answers from an agronomist-verified corpus first, uses a vision LLM only when the corpus is not confident, and never recommends a product outside its registered label claim. Public lab datasets (PlantVillage) lose 20–50 accuracy points on real field photos, so the corpus is built from FCC's own field images.

```
flowchart TD
  A[Photo from app] --> B[On-device quality check<br/>blur, leaf present,
crop guess]
  B -->|fails| R[Retake tip]
  B -->|passes| C[Upload + queue]
  C --> D[Embed image<br/>vision encoder]
  D --> E[pgvector kNN<br/>top 20 corpus matches]
  E --> F{Match score and<br/>vote agreement}
  F -->|high| G[Corpus answer]
  F -->|medium or low| H[Vision LLM with<br/>crop, region, season, top
matches]
  H --> I[Provisional answer<br/>+ agronomist queue]
  G --> J[Treatment + label-claim filter<br/>+ own-brand ranking]
  I --> J
  J --> K[Result to farmer / officer]
  I --> L[Agronomist confirms or corrects]
  L --> M[Corpus + retraining set]
```

The on-device step keeps bad photos off the network; everything after upload runs asynchronously so a slow village connection never blocks the user.

3.1 Requirements

ID	Requirement	Pri	Acceptance criteria
DX-001	On-device TFLite model (< 8 MB): image quality, leaf/plant present, crop class for top 6 crops	P0	Runs under 400 ms on a 2 GB RAM device

ID	Requirement	Pri	Acceptance criteria
DX-002	Server embedding with a pinned vision encoder (e.g. SigLIP or DINOv2 family), 768-dim vectors stored in pgvector (HNSW index)	P0	kNN over 200k vectors under 150 ms p95
DX-003	Decision rule: top-1 cosine ≥ threshold T1 and ≥ 60% of top-10 agree on problem → "high"; tunable per crop	P0	Thresholds stored in config, versioned
DX-004	LLM fallback: send image + crop + district + season + top-5 corpus candidates; LLM must choose from candidates or return "unknown"; structured JSON output	P0	Invalid JSON retried once, then "unknown"
DX-005	Every result stores: model versions, corpus version, scores, candidates, final label, source (corpus/LLM/agronomist)	P0	Reproducible from stored data
DX-006	Agronomist review SLA: provisional results reviewed within 24 h in season	P0	Queue ageing alert at 18 h
DX-007	Treatment engine: problem → treatments (organic, chemical, cultural) → active ingredients → SKUs with valid label claim for that crop and pest	P0	Unit tests prove no SKU without claim is ever returned
DX-008	Ranking: valid SKUs ordered by efficacy tier (agronomist), then own-brand, then in stock at nearest store, then price	P0	Own brand never outranks a higher efficacy tier
DX-009	Dosage per acre and per 15 L tank, pre-harvest interval, PPE and safety text from the label	P0	Shown on every chemical recommendation
DX-010	Out-of-scope handling: unsupported crop or non-plant image returns a friendly message and routes to expert	P0	No hallucinated diagnosis in test set of non-plant images

ID	Requirement	Pri	Acceptance criteria
DX-011	Nightly evaluation job on a held-out, agronomist-labelled field test set; metrics per crop and problem	P1	Report in console; alert if top-1 accuracy drops > 5 points
DX-012	Retraining pipeline: monthly fine-tune of the on-device crop model and re-embedding when the encoder changes	P1	Vertex AI pipeline run with approval gate
DX-013	Outbreak detection: cluster confirmed diagnoses by district and week; trigger FM-009 alerts	P1	Rule fires on seeded test data

3.2 Corpus entry

Field	Notes
problem_id, crop, problem_type	disease, pest, nutrient, abiotic
names	Local-language names per supported language
symptoms	Short checklist the farmer can confirm
images	Field images with stage, part (leaf, stem, fruit), severity, region, season
treatments	Linked to active ingredients and SKUs via label claims
status	draft → reviewed → approved → retired; only approved is served
provenance	Who labelled, who approved, when

3.3 Seeding plan

1. Start with the 6 pilot crops and the 10–15 most common problems each (about 80 problems).
2. Seed with licensed public field datasets for bootstrapping only; mark as `source=public`.

3. Field officers photograph every scouting case during Rabi 2026; agronomist labels weekly.
4. Target 150+ approved field images per problem before marking it "high confidence capable".

3.4 Compliance guardrails

- Recommendations must match a CIB&RC-registered label claim for the crop and pest; enforced in code (DX-007) and in the product master (WC-005).
- Every chemical recommendation shows dosage, safety interval and PPE.
- Provisional results never show a product "buy" button until confirmed by corpus or agronomist; they show "talk to your field officer" instead.
- Prompt and output logs for the LLM fallback are retained 1 year for audit.

4. Visit verification and location

Visits are verified by a proof-of-visit bundle, not by hidden tracking. Location is collected only on duty, disclosed up front, and tied to travel allowance so officers benefit from keeping it on. Google Play forbids hidden employee tracking and requires prominent disclosure for background location; this design satisfies both.

```
sequenceDiagram
  participant O as Field officer app
  participant S as API
  participant F as Farmer (SMS or app)
  O->>S: Check-in (GPS, accuracy, time, device integrity)
  S-->>O: Inside geofence? yes / no + reason
  O->>S: Photos (in-app camera) + notes + purpose
  O->>S: Request farmer confirmation
  S->>F: OTP by SMS or push prompt
  F-->>S: OTP entered or tap confirm
  S-->>O: Visit confirmed
  O->>S: Check-out (duration computed)
  S->>S: Score visit, run validation agents
```

4.1 Requirements

ID	Requirement	Pri	Acceptance criteria
VV-001	On-duty location sampling: every 2 min moving, every 10 min stationary, using fused provider; batch upload every 15 min or on Wi-Fi/charging	P0	Battery use < 6% over an 8-hour shift on reference device
VV-002	Store accuracy, speed, battery, mock-location flag, and device clock vs server clock drift with each point	P0	Fields present on every point
VV-003	Geofence per farm from registration polygon (buffer default 150 m) and per village meeting point	P0	Geofence test passes on sample polygons
VV-004	Visit score 0–100: geofence (30), farmer confirmation (30), in-app photo with matching EXIF time and GPS (20), duration ≥ minimum (10), device integrity (10)	P0	Score components stored and shown in console
VV-005	Visit status: verified (≥ 70), partial (40–69), unverified (< 40); only verified visits count toward incentives by default	P0	Incentive engine uses status
VV-006	Track processing: snap to roads, compute km, stop detection (≥ 5 min within 100 m), gaps (> 30 min no data while on duty)	P1	Stops and gaps visible in day replay
VV-007	Auto end of duty at configurable time (default 20:00) or after 60 min idle at home location, with prompt	P0	No tracking beyond auto end
VV-008	Officer sees own day timeline and km; can add a reason for gaps (network, personal break)	P1	Reason stored and shown to cluster head

ID	Requirement	Pri	Acceptance criteria
VV-009	Play Store background-location declaration, demo video, and privacy policy prepared; app flagged as monitoring tool only if required by review	P0	Checklist in release PR
VV-010	Written employee location policy signed in HR onboarding; in-app acceptance references the same version	P0	Policy version matches

4.2 Why officers keep it on

- Travel allowance is paid from computed km (FA-023); no track, no automatic TA.
- Incentives count verified visits (VV-005).
- Disputes resolve in the officer's favour when evidence exists ("I was there" is provable).
- The officer sees the same data the manager sees about their own day (VV-008).

5. Analytics layer

The owner gets one console home that answers three questions each morning: which officers are really working, which farmers are growing or slipping, and which stores are making money. Operational data flows from Postgres to BigQuery by change-data-capture; dbt builds governed models; the console and Looker Studio read the same models.

```
flowchart LR
  PG[(Cloud SQL<br/>Postgres)] --> DS[Datastream CDC]
  DS --> RAW[(BigQuery raw)]
  RAW --> DBT[dbt: staging → marts]
  DBT --> MART[(BigQuery marts)]
  MART --> WC[Web console dashboards]
  MART --> LS[Looker Studio<br/>owner reports]
  MART --> VA[Validation agents]
```

5.1 Dashboards

ID	Dashboard	Audience	Content
AN-001	Owner home	Owner, directors	Revenue, own-brand share, credit outstanding and overdue, verified visits, active farmers, open validation flags; vs target and last year; store map coloured by performance
AN-002	Field team scorecard	Owner, cluster head	Per officer: verified visits/day, unique farmers reached, new registrations, confirmation rate, avg time on farm, km/day, conversion (visited → bought in 30 days), collections, flags; rank and trend
AN-003	Officer day replay	Cluster head, auditor	Map track, stops, visits with scores, gaps with reasons
AN-004	Farmer 360 and segments	Owner, cluster head	Farmers by crop, acreage, district, stage; lifetime value, last visit, last purchase, credit risk, app usage; churn risk list
AN-005	Store performance	Owner, store manager (own)	Sales by category and brand, margin, stock turns, stockouts, day-close variance, credit book ageing
AN-006	Crop health map	Owner, agronomist	Confirmed problems by district and week; outbreak alerts
AN-007	Diagnosis quality	Agronomist, product	Share answered by corpus, escalation rate, review SLA, accuracy on test set, top unknowns
AN-008	Campaign ROI	Owner, marketing	Sent, delivered, opened, orders, revenue, own-brand lift per campaign

ID	Dashboard	Audience	Content
AN-009	Data quality	Owner, admin	Validation flag volume by type, resolution time, repeat offenders

5.2 KPI definitions

KPI	Definition
Verified visit	Visit with score ⩾ 70 (VV-004)
Unique farmers reached	Distinct farmers with ⩾ 1 verified visit in period
Confirmation rate	Visits confirmed by farmer ÷ all visits
Visit conversion	Farmers with a purchase within 30 days after a verified visit ÷ farmers visited
Own-brand share	Own-brand net sales ÷ total net sales
Active farmer	Purchase, verified visit, or app session in last 60 days
Credit overdue	Khata balance past due date; buckets 0–30, 31–60, 61–90, 90+ days
Farmer LTV	Cumulative gross margin from the farmer
Time on farm	Sum of visit durations (check-in to check-out) for verified visits

5.3 Requirements

ID	Requirement	Pri	Acceptance criteria
AN-010	CDC latency under 15 minutes from Postgres to BigQuery marts for operational dashboards	P1	Freshness test in dbt
AN-011	All dashboards filter by tenant, date range, cluster, store, officer, crop	P0	Filters persist in URL

ID	Requirement	Pri	Acceptance criteria
AN-012	dbt tests (unique, not_null, relationships, accepted_values) on every mart	P0	dbt test green in CI
AN-013	Row-level security in BigQuery by tenant and role (authorised views)	P0	Cluster head sees only own cluster
AN-014	PII minimisation: marts use farmer_id and masked phone; full PII only in console detail views with audit	P0	Masking verified
AN-015	Weekly owner email: 1-page PDF of AN-001 and top 5 flags	P1	Delivered Monday 8:00 IST
AN-016	Natural-language questions over marts for owners ("which officers had fewer than 3 verified visits last week?") via an LLM with read-only SQL on authorised views	P2	Queries logged; no write access

For P0, the console reads read-only Postgres replicas for live figures; BigQuery and dbt arrive in P1 when volume justifies it.

6. Validation agents

Validation agents are scheduled and event-driven checks that flag suspicious or low-quality farmer and employee data, attach the evidence, and route each flag to a human owner. They never penalise anyone automatically: a flag is a question for a cluster head or auditor, not a verdict.

6.1 How they run

Layer	What it does	Runs on
Inline rules	Cheap checks at write time (format, duplicates, geofence)	API request path
Event agents	Checks triggered by a new visit, registration, sale or payment	Pub/Sub → Cloud Run worker
Batch agents	Cross-record patterns over days or weeks	Cloud Scheduler → Cloud Run Jobs (nightly 02:00 IST)

Layer	What it does	Runs on
Review agent	LLM that reads a flag's evidence bundle, writes a 3-line plain summary and a suggested action for the reviewer	Cloud Run worker, LLM API

```
stateDiagram-v2
  [*] --> Open
  Open --> InReview: reviewer opens
  InReview --> Resolved_valid: data confirmed OK
  InReview --> Resolved_fixed: data corrected
  InReview --> Escalated: needs auditor or HR
  Escalated --> Resolved_fixed
  Escalated --> Resolved_action: action taken
  Resolved_valid --> [*]
  Resolved_fixed --> [*]
  Resolved_action --> [*]
```

Every resolution is fed back as a label so rule thresholds can be tuned and false positives drop over time.

6.2 Farmer data checks

ID	Check	Type	Severity
VA-F01	Phone format, OTP-verified, not in blocklist of staff phones	Inline	High
VA-F02	Duplicate farmer: same phone, or fuzzy name + village + farm centroid within 50 m	Inline + batch	Medium
VA-F03	Farm polygon sanity: area vs declared acres within ±30%; no self-intersection; not overlapping another farmer's farm > 20%	Inline	Medium
VA-F04	Farm location plausibility: within tenant service districts; not on road, water body, or built-up area (land-cover layer)	Batch	Medium
VA-F05	Crop plausibility: crop and sowing date consistent with season and district crop calendar	Batch	Low

ID	Check	Type	Severity
VA-F06	Registration burst: officer registers > N farmers in < M minutes, or many farmers sharing one device/phone	Event	High
VA-F07	Ghost farmer: registered but no confirmation, no purchase, no app login within 60 days	Batch	Medium
VA-F08	Photo reuse: perceptual hash of farm or visit photo matches another farmer's photo	Event	High
VA-F09	Consent missing or withdrawn but farmer in a campaign segment	Batch	High
VA-F10	Credit risk: khata overdue > 90 days, or new credit while overdue	Batch	Medium

6.3 Employee data checks

ID	Check	Type	Severity
VA-E01	Mock location, rooted device, or clock drift > 5 min during duty	Event	High
VA-E02	Impossible travel: speed between consecutive points > 90 km/h on rural roads	Event	High
VA-E03	Visits clustered at one spot (≥ 5 farms' visits checked in within 200 m of each other outside a meeting)	Batch	High
VA-E04	Short visits: > 40% of a day's visits under minimum duration	Batch	Medium
VA-E05	Confirmation anomalies: same OTP-receiving phone confirms visits for several farmers; confirmations always from one device	Batch	High
VA-E06	Duty with no activity: > 3 h on duty with no visit, stop, or movement	Batch	Low
VA-E07	TA claim vs computed km differs by > 15%	Event	Medium

ID	Check	Type	Severity
VA-E08	Photo integrity: in-app photo EXIF time/GPS inconsistent with check-in; duplicate photos across visits	Event	High
VA-E09	Sales anomalies: discounts above policy, sales to self or relatives' phones, frequent cancellations after incentive cut-off	Batch	High
VA-E10	Cash variance at day close above threshold; repeated small variances	Event + batch	Medium
VA-E11	Performance drift: verified visits down > 30% week-on-week for 2 weeks	Batch	Low (coaching)

6.4 Requirements

ID	Requirement	Pri	Acceptance criteria
VA-001	Rules defined as versioned config (YAML) with thresholds per tenant; no code change to tune	P0	Threshold change deployed via config PR
VA-002	Each flag stores: rule id and version, subject (farmer/employee/visit), evidence (records, map snapshot, photos), severity, owner, SLA	P0	Evidence renders in WC-010
VA-003	Routing: employee flags to cluster head; high severity also to auditor; farmer data flags to store manager	P0	Routing tests
VA-004	Review agent summary uses only evidence in the bundle; cites record ids; no access to other data	P1	Summary contains only referenced ids
VA-005	Employee sees flags about themselves once resolved, with the outcome, and can add an explanation while open	P1	Officer view shows own flags

ID	Requirement	Pri	Acceptance criteria
VA-006	Precision tracking per rule (valid vs false positive); rules with precision < 30% over 4 weeks auto-disabled pending review	P1	Weekly rule-quality report
VA-007	Golden test fixtures for every rule (a positive and a negative case) run in CI	P0	Rule without fixtures fails CI

7. Non-functional requirements

The apps must work on cheap Android phones with patchy 2G/3G, in five languages, and keep personal data inside India under DPDP rules.

ID	Area	Requirement
NF-001	Devices	Android 8+ (API 26), 2 GB RAM, 16 GB storage; tested on 3 reference devices (one Samsung A-series, one Redmi, one low-end Realme/Itel)
NF-002	Offline	Field agent app fully usable offline for 72 h; farmer app caches last results, orders and calendar
NF-003	Sync	Outbox pattern with idempotency keys; resumable image uploads (chunked, 256 KB); retry with backoff
NF-004	Image budget	Photos resized on device to 1280 px long edge, JPEG q80 (~150–250 KB)
NF-005	Performance	App cold start < 3 s on reference low-end device; API p95 < 300 ms for reads, < 600 ms for writes; diagnosis corpus answer < 5 s end to end on 3G
NF-006	Availability	API 99.5% monthly (pilot), 99.9% (SaaS); RPO 15 min, RTO 4 h
NF-007	Scale target	50 tenants, 5,000 officers, 2 million farmers, 20 million photos without re-architecture

ID	Area	Requirement
NF-008	Localisation	Five launch languages; numbers in Indian format (1,00,000); dates DD-MM-YYYY; units acre and litre; all farmer-facing text reviewed by a native speaker
NF-009	Accessibility	Minimum 16 sp body text, 48 dp touch targets, icons always paired with text, audio playback for key farmer instructions, WCAG AA contrast
NF-010	Literacy	Farmer flows usable with icons, photos and audio; no flow requires typing beyond phone number and OTP
NF-011	Data residency	All personal data stored and processed in asia-south1 / asia-south2; LLM calls send images and crop context only, never names or phones
NF-012	Retention	Raw location points 180 days; visit records 7 years; photos 3 years then archived; audit logs 1 year minimum; financial records 8 years
NF-013	DPDP	Notice + consent flows for farmers; legitimate-use basis documented for employees; data-principal rights (access, correction, erasure, grievance) in-app and via console; breach notification runbook; consent records exportable
NF-014	Observability	Structured JSON logs with trace id; OpenTelemetry traces across app → API → workers
NF-015	Cost	Pilot infra (5 stores) under ₹25,000/month; at 50 stores under ₹1,00,000/month excluding SMS and LLM; cost dashboard per tenant
NF-016	Maintainability	80% line coverage for API domain logic; typed everywhere (mypy strict, TypeScript strict, Dart strict analysis)

Cost figures are planning estimates; validate with the GCP pricing calculator once traffic assumptions are confirmed.

8. Architecture and data model

One modular-monolith API on Cloud Run plus async workers, one Postgres database, and object storage; split services only when load or team size demands it. This keeps cost near zero when idle and the codebase simple for a small team.

```
flowchart TD
  subgraph Clients
    AG[Field agent app]
    FA[Farmer app]
    WEB[Web console]
  end
  AG --> LB[HTTPS LB + Cloud Armor]
  FA --> LB
  WEB --> LB
  LB --> API[api — FastAPI<br/>Cloud Run]
  LB --> CON[console — Next.js<br/>Cloud Run]
  API --> SQL[(Cloud SQL Postgres<br/>PostGIS + pgvector)]
  API --> GCS[(Cloud Storage<br/>photos, exports)]
  API --> PS[Pub/Sub topics]
  PS --> WK[worker — Cloud Run<br/>diagnosis, validation, notify]
  WK --> SQL
  WK --> EXT[Razorpay, SMS, WhatsApp,<br/>LLM API, Maps]
  SQL --> BQ[(BigQuery via Datastream)]
```

The console calls the API like any other client; it holds no direct database access.

8.1 Services and modules

Deployable	Modules inside	Scaling
`api`	auth, tenants, users, farmers, farms, visits, location, catalog, orders, payments, khata, stock, diagnosis (sync part), notifications, flags, reports	Cloud Run, min 1 instance in prod (cold starts), max 20
`worker`	diagnosis pipeline, validation agents (event), notification fan-out, track processing, webhook processing	Cloud Run, Pub/Sub push, min 0

Deployable	Modules inside	Scaling
`jobs`	nightly validation batch, evaluation, exports, retention purge, beat plan generation	Cloud Run Jobs + Cloud Scheduler
`console`	Next.js web console	Cloud Run, min 0 (1 in prod)
`embed`	Vision encoder for embeddings (GPU optional)	Cloud Run with L4 GPU at scale, CPU in pilot

8.2 Core entities

Entity	Key columns (all have id uuid, tenant_id, created_at, updated_at, deleted_at, version)
tenant	name, plan, branding jsonb, settings jsonb
user	phone, name, role, store_id, cluster_id, language, status, device_id
store / cluster	name, district, state, location geography(Point), geofence
farmer	phone, name, village_id, language, consent_marketing, consent_version, registered_by, source
farm	farmer_id, boundary geography(Polygon), area_acres, soil, irrigation, ownership
crop_plot	farm_id, crop_id, variety, season, sowing_date, stage, expected_harvest
visit	officer_id, farmer_id, farm_id, purpose, checkin_at, checkout_at, checkin_point, score, status, confirmation_channel
visit_photo	visit_id, gcs_uri, phash, exif_time, exif_point, taken_in_app
location_point	officer_id, ts, point, accuracy_m, speed, battery, is_mock, clock_drift_s (partitioned by day)
duty_session	officer_id, start_at, end_at, km, stops jsonb

Entity	Key columns (all have id uuid, tenant_id, created_at, updated_at, deleted_at, version)
diagnosis	subject (farmer/officer), crop_plot_id, images, result jsonb, source, confidence, model_versions, corpus_version, status
corpus_problem / corpus_image	see section 3.2; corpus_image.embedding vector(768)
product / sku	name, brand, own_brand, category, hsn, gst_rate, pack_size, mrp
label_claim	sku_id, crop_id, pest_or_disease_id, dose_per_acre, phi_days, source_ref
stock_batch	store_id, sku_id, batch_no, expiry, qty
order / order_line	farmer_id, store_id, channel, status, totals
payment	order_id or khata_id, mode, amount, provider_ref, status
khata_entry	farmer_id, store_id, type (debit/credit), amount, due_date, ref
flag	rule_id, rule_version, subject_type, subject_id, severity, evidence jsonb, owner_id, status, resolution
audit_log	actor_id, action, entity, entity_id, before, after, ip, device
consent_record	subject, purpose, version, granted_at, withdrawn_at, channel

Money is stored as integer paise. All timestamps are UTC in the database and shown in IST.

8.3 API conventions

- OpenAPI 3.1 spec in `apps/api/openapi.yaml` is the contract; Dart and TypeScript clients are generated from it in CI.
- REST, JSON, `/v1/...`; cursor pagination; `Idempotency-Key` header on every POST from mobile.

- Sync endpoint: `POST /v1/sync` accepts a batch of outbox operations and returns server changes since a cursor.
- Errors follow RFC 9457 problem+json with a stable `code`.
- Webhooks (Razorpay) verified by signature, processed via Pub/Sub, idempotent by provider event id.

8.4 Mobile architecture

- Flutter monorepo with `packages/core` (models, API client, sync engine, design system widgets, i18n) shared by `apps/agent` and `apps/farmer`.
- State: Riverpod; local DB: Drift (SQLite); background work: WorkManager; location: foreground service plugin with fused provider.
- Crash and performance: Firebase Crashlytics and Performance; remote config for feature flags and thresholds.

9. GCP infrastructure as code

All infrastructure is Terraform in `infra/`, one GCP project per environment (`farmerapp-dev`, `farmerapp-stg`, `farmerapp-prd`) plus a shared `farmerapp-ops` project for state, artifact registry and CI identity. Nothing is created by hand in the console except billing and the org.

9.1 Layout

```
infra/
  bootstrap/            # run once: ops project, state bucket, WIF pool, CI
service accounts
  modules/
    network/            # VPC, subnets, Serverless VPC connector / Direct VPC
egress, Cloud NAT, PSA for Cloud SQL
    sql/                # Cloud SQL Postgres 16, flags, backups, read replica
(prd)
    run_service/        # generic Cloud Run v2 service (image, env, secrets,
scaling, SA)
    run_job/            # Cloud Run Job + Cloud Scheduler trigger
    pubsub/             # topics, push subscriptions, DLQs
    storage/            # buckets with lifecycle rules, CMEK optional
    lb/                 # HTTPS LB, managed certs, serverless NEGs, Cloud
Armor policy
    secrets/            # Secret Manager secrets (values set out-of-band)
    analytics/          # BigQuery datasets, Datastream, authorised views
    monitoring/         # uptime checks, alert policies, dashboards,
notification channels
    iam/                # per-service service accounts and least-privilege
bindings
  envs/
    dev/  stg/  prd/    # main.tf wiring modules, terraform.tfvars,
backend.tf
```

9.2 Resources per environment

Resource	dev	stg	prd
Cloud SQL tier	db-g1-small (or shared-core)	db-custom-1-3840	db-custom-2-7680, HA regional, read replica
Cloud Run min instances (api/console)	0 / 0	0 / 0	1 / 1
Backups	7 days	7 days	30 days + PITR, cross-region copy to asia-south2
Cloud Armor	off	on (log only)	on (enforce)
Datastream → BigQuery	off	on	on

Resource	dev	stg	prd
Deletion protection	off	on	on

PubSub topics: `visit.created`, `farmer.registered`, `diagnosis.requested`, `payment.webhook`, `order.created`, `location.batch`, `notify.send`; each with a dead-letter topic and 7-day retention. Buckets: `photos` (Standard → Nearline at 90 days → Archive at 3 years), `exports` (delete at 30 days), `corpus` (versioned), `tfstate` (versioned, in ops project).

9.3 Example: backend and providers

```
terraform {
  required_version = ">= 1.8"
  required_providers {
    google      = { source = "hashicorp/google", version = ">= 6.0" }
    google-beta = { source = "hashicorp/google-beta", version = ">= 6.0" }
  }
  backend "gcs" {
    bucket = "farmerapp-ops-tfstate"
    prefix = "envs/prd"
  }
}

provider "google" {
  project = var.project_id
  region  = "asia-south1"
}
```

9.4 Example: Cloud SQL

```
resource "google_sql_database_instance" "main" {
  name                = "farmerapp-${var.env}"
  database_version    = "POSTGRES_16"
  region              = "asia-south1"
  deletion_protection = var.env != "dev"

  settings {
    tier              = var.sql_tier
    availability_type = var.env == "prd" ? "REGIONAL" : "ZONAL"
    disk_autoresize   = true
```

```
    ip_configuration {
      ipv4_enabled    = false
      private_network = var.vpc_id
      ssl_mode        = "ENCRYPTED_ONLY"
    }
    backup_configuration {
      enabled                        = true
      point_in_time_recovery_enabled = var.env == "prd"
      transaction_log_retention_days = 7
      backup_retention_settings { retained_backups = var.env == "prd" ? 30 :
7 }
    }
    database_flags {
      name  = "cloudsql.iam_authentication"
      value = "on"
    }
    database_flags {
      name  = "cloudsql.logical_decoding"   # needed for Datastream CDC
      value = "on"
    }
    insights_config { query_insights_enabled = true }
  }
}
```

Extensions `postgis`, `vector` and `pg_trgm` are created by the first Alembic migration, not by Terraform.

9.5 Example: Cloud Run service module

```
resource "google_cloud_run_v2_service" "svc" {
  name     = var.name
  location = "asia-south1"
  ingress  = "INGRESS_TRAFFIC_INTERNAL_LOAD_BALANCER"

  template {
    service_account = var.service_account_email
    scaling {
      min_instance_count = var.min_instances
      max_instance_count = var.max_instances
    }
    vpc_access {
      network_interfaces {
```

```
        network    = var.vpc_name
        subnetwork = var.subnet_name
      }
      egress = "PRIVATE_RANGES_ONLY"
    }
    containers {
      image = var.image   # set by CI; Terraform ignores later image changes
      resources { limits = { cpu = var.cpu, memory = var.memory } }
      dynamic "env" {
        for_each = var.secret_env
        content {
          name = env.key
          value_source {
            secret_key_ref {
              secret  = env.value
              version = "latest"
            }
          }
        }
      }
    }
  }
  lifecycle { ignore_changes = [template[0].containers[0].image] }
}
```

9.6 Example: GitHub Actions identity (no keys)

```
resource "google_iam_workload_identity_pool" "gh" {
  workload_identity_pool_id = "github"
}

resource "google_iam_workload_identity_pool_provider" "gh" {
  workload_identity_pool_id          =
google_iam_workload_identity_pool.gh.workload_identity_pool_id
  workload_identity_pool_provider_id = "github-oidc"
  attribute_mapping = {
    "google.subject"       = "assertion.sub"
    "attribute.repository" = "assertion.repository"
    "attribute.ref"        = "assertion.ref"
  }
  attribute_condition = "assertion.repository == \"<org>/farmerapp\""
  oidc { issuer_uri = "https://token.actions.githubusercontent.com" }
}
```

Separate deployer service accounts per environment; the `prd` deployer may only be impersonated from the `main` branch with the `production` GitHub environment approval.

9.7 IaC rules

- `terraform fmt`, `validate`, `tflint` and `checkov` run on every PR touching `infra/`; the PR shows the `plan` as a comment.
- `apply` runs only from CI: dev on merge, stg and prd on manual approval.
- Weekly drift detection job runs `plan` on every env and opens an issue if non-empty.
- No secrets in Terraform state: secrets are created empty and values are added via `gcloud secrets versions add` by an admin.

10. Repository, Git workflow and CI/CD

One GitHub monorepo, trunk-based development with short-lived branches, and GitHub Actions that test, scan, build and deploy every merge to dev automatically; staging and production need a human approval.

10.1 Repository layout

```
farmerapp/
  CLAUDE.md                    # instructions for Claude Code (section 14.2)
  docs/SPEC.md                 # this document
  docs/adr/                    # architecture decision records
  design/                      # Claude Design export: tokens.json, screens/,
components/
  apps/
    api/                       # FastAPI, SQLAlchemy 2, Alembic, pytest
    worker/                    # shares api domain package; Pub/Sub handlers
    jobs/                      # batch jobs, validation agents (batch)
    console/                   # Next.js 15, TypeScript, Tailwind, shadcn/ui,
TanStack Query
    mobile/
      packages/core/           # models, generated API client, sync, design
system, i18n
      apps/agent/              # field agent app (+ store mode)
      apps/farmer/             # farmer app
  ml/
    ondevice/                  # crop + quality model training, TFLite export
    embed/                     # encoder service
    eval/                      # test sets, evaluation scripts
  rules/                       # validation agent rules (YAML) + fixtures
  dbt/                         # analytics models
  infra/                       # Terraform (section 9)
  .github/workflows/
```

10.2 Git rules

- `main` is always deployable; branches `feat/FA-012-geofence`, `fix/...`, `chore/...`; branches live under 2 days where possible.
- Conventional Commits with requirement ID: `feat(FA-012): capture farm boundary`.
- Pull requests: template with requirement IDs, screenshots for UI, migration notes, test evidence; at least 1 review (2 for `infra/`, `rules/`, payments and auth code via CODEOWNERS).
- Branch protection on `main`: required checks green, linear history (squash merge), signed commits, no force push.
- Every DB change is an Alembic migration; migrations are backward compatible (expand → migrate → contract across two releases).

- Feature flags (Firebase Remote Config for mobile, config table for API) wrap incomplete features; no long-running feature branches.
- Semantic versioning per deployable; mobile `versionCode` derived from CI run number.

10.3 Pipelines

Workflow	Trigger	Steps
`ci-api.yml`	PR touching api/worker/jobs/rules	ruff, mypy strict, pytest with Postgres+PostGIS service container, coverage ≥ 80%, OpenAPI diff check, rule fixtures, bandit, pip-audit
`ci-console.yml`	PR touching console	eslint, tsc, vitest, Playwright smoke against mocked API, npm audit, bundle size check
`ci-mobile.yml`	PR touching mobile	flutter analyze, dart format check, unit + widget tests, golden tests for design system, build debug APKs, APK size check (farmer < 25 MB)
`ci-infra.yml`	PR touching infra	fmt, validate, tflint, checkov, plan per env posted as PR comment
`ci-dbt.yml`	PR touching dbt	dbt build against dev BigQuery dataset, tests
`security.yml`	PR + nightly	gitleaks, Semgrep, Trivy (images and IaC), OSV scanner, license check
`deploy.yml`	merge to main	build images with Cloud Build or buildx → Artifact Registry (tag = git SHA) → run migrations as Cloud Run Job → deploy dev → smoke tests → await approval → stg → e2e → await approval → prd with 10% traffic canary for 30 min → 100%
`mobile-release.yml`	tag `agent-v*` / `farmer-v*`	build signed AAB (keys in Secret Manager via WIF) → Play internal track → promote to closed/production manually in Play Console

Workflow	Trigger	Steps
`drift.yml`	weekly	terraform plan all envs, open issue on drift

10.4 Deploy workflow skeleton

```
name: deploy
on:
  push:
    branches: [main]
permissions:
  contents: read
  id-token: write
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      image: ${{ steps.meta.outputs.image }}
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: ${{ vars.WIF_PROVIDER }}
          service_account: ${{ vars.BUILD_SA }}
      - uses: google-github-actions/setup-gcloud@v2
      - id: meta
        run: echo "image=asia-south1-docker.pkg.dev/farmerapp-
ops/apps/api:${GITHUB_SHA}" >> "$GITHUB_OUTPUT"
      - run: gcloud builds submit apps/api --tag "${{
steps.meta.outputs.image }}"

  deploy-dev:
    needs: build
    runs-on: ubuntu-latest
    environment: dev
    steps:
      - uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: ${{ vars.WIF_PROVIDER }}
          service_account: ${{ vars.DEPLOY_SA_DEV }}
      - uses: google-github-actions/setup-gcloud@v2
```

```
      - run: gcloud run jobs execute migrate --region asia-south1 --wait --
project farmerapp-dev
      - uses: google-github-actions/deploy-cloudrun@v2
        with:
          service: api
          region: asia-south1
          project_id: farmerapp-dev
          image: ${{ needs.build.outputs.image }}
      - run: ./scripts/smoke.sh https://api.dev.farmerapp.in

  deploy-prd:
    needs: [build, deploy-dev]   # add deploy-stg the same way
    runs-on: ubuntu-latest
    environment: production       # required reviewers configured in GitHub
    steps:
      - run: echo "same as dev with DEPLOY_SA_PRD, --no-traffic then gradual
traffic shift"
```

The migrate job must use the new image before the service rolls; in the full workflow, update the job's image first (`gcloud run jobs update migrate --image ...`).

10.5 Development cycle

```
flowchart LR
  A[Pick requirement ID] --> B[Branch + tests first]
  B --> C[Implement]
  C --> D[Local checks<br/>make check]
  D --> E[PR + CI + review]
  E --> F[Merge → dev auto-deploy]
  F --> G[Staging approval<br/>e2e + QA script]
  G --> H[Prod approval<br/>canary → 100%]
  H --> I[Monitor 24 h<br/>close requirement]
```

- Local environment: `docker compose up` starts Postgres+PostGIS+pgvector, Pub/Sub emulator, fake GCS, and a mock SMS/payment server; `make seed` loads a demo tenant with 5 stores, 5 officers, 200 farmers.
- `make check` runs the same lint, type and test commands as CI.
- Release cadence: API and console continuously; mobile every 2 weeks (agent) and every 4 weeks (farmer) during season, with hotfix path.
- Season freeze: no risky releases in the first 3 weeks of Kharif and Rabi sowing peaks; flags only.

11. Security checks and controls

Security is enforced in four places: the pipeline blocks known-bad code and dependencies, the platform limits blast radius, the app protects data on lost phones, and scheduled reviews catch what automation misses. The baseline is OWASP ASVS Level 2 for the API and console and OWASP MASVS-L1 (L2 for payments and auth flows) for mobile.

11.1 Pipeline checks (block merge on failure)

ID	Check	Tool
SEC-001	Secret scanning in commits and PRs	gitleaks + GitHub secret scanning with push protection
SEC-002	Static analysis (injection, authz, crypto misuse)	Semgrep (p/python, p/typescript, p/owasp-top-ten), bandit
SEC-003	Dependency vulnerabilities (high/critical block)	pip-audit, npm audit, OSV-Scanner, Dependabot/Renovate weekly
SEC-004	Container image vulnerabilities	Trivy on built images; distroless or slim base images; non-root user
SEC-005	IaC misconfiguration	checkov + tflint
SEC-006	Authorisation tests: every endpoint has a test for each role (allowed/denied) and cross-tenant access	pytest parametrised matrix generated from OpenAPI
SEC-007	License compliance (no AGPL in shipped apps)	license checker
SEC-008	Mobile: no secrets in APK, no debuggable release, obfuscation on	MobSF scan in `mobile-release.yml`

11.2 Platform controls

ID	Control
SEC-010	One service account per deployable with least-privilege roles; no default compute SA; no user-managed JSON keys (org policy `iam.disableServiceAccountKeyCreation`)
SEC-011	Cloud SQL private IP only, IAM database auth, TLS enforced; app DB user has no DDL rights (migrations use a separate role)
SEC-012	Postgres row-level security keyed on `app.tenant_id` set per request; tested in CI
SEC-013	All secrets in Secret Manager; rotated every 90 days (OTP provider, Razorpay, LLM keys)
SEC-014	Cloud Armor: OWASP CRS rules, rate limits (OTP send 5/min/IP, 3/15 min/phone; login 10/min/IP), geo allow-list India for farmer/agent APIs, bot management on OTP endpoints
SEC-015	Cloud Run ingress only from the load balancer; console behind IAP or SSO for staff roles in P1
SEC-016	Buckets private, uniform access, signed URLs (15 min) for photo upload and view
SEC-017	Audit logs (Admin Activity + Data Access for Cloud SQL, Secret Manager, GCS) exported to a locked log bucket, 400-day retention
SEC-018	Organisation policies: restrict resource locations to India regions, disable public IPs on SQL, require OS login, domain-restricted sharing
SEC-019	Security Command Center (standard) enabled; findings routed to the security channel

11.3 Application controls

ID	Control
SEC-020	JWT access tokens 15 min, refresh 30 days with rotation and reuse detection; device-bound refresh for field agents
SEC-021	OTP: hashed at rest, constant-time compare, single use; SMS pumping protection (per-country block, velocity rules)
SEC-022	Input validation with Pydantic models; output encoding in console; strict CSP
SEC-023	Payment webhooks verified by HMAC signature and replay window; amounts always recomputed server-side
SEC-024	Mobile local DB encrypted (SQLCipher via Drift), keys in Android Keystore; certificate pinning for API domain; Play Integrity API check on login and sensitive actions
SEC-025	Remote wipe of local data when a user is disabled or the device is reported lost
SEC-026	PII field-level encryption for phone numbers and addresses (Cloud KMS envelope) with a searchable hashed phone column
SEC-027	LLM calls: no PII in prompts; image stripped of EXIF location before sending; prompts and outputs logged; output schema validated
SEC-028	Admin actions (role change, export, settings) require re-auth and are audited

11.4 Scheduled reviews

- Before pilot and before SaaS launch: external penetration test (API, console, both apps).

- Quarterly: access review of all staff and service accounts; secret rotation check; restore drill from backup.
- Monthly: dependency and base-image refresh; review of Cloud Armor blocks and SCC findings.
- Incident response runbook with DPDP breach notification steps, owner and contact list, tested once a year.

12. Monitoring, alerting and operational workflows

Three kinds of monitoring run side by side: platform health (is it up and fast), business health (are officers syncing, are payments landing), and data health (are validation agents and pipelines running). Every alert has an owner, a severity and a runbook link; everything is defined in Terraform (`infra/modules/monitoring`).

12.1 Service level objectives

SLO	Target	Alert when
API availability (non-5xx)	99.5% over 28 days	Burn rate > 14× over 1 h or > 6× over 6 h
API latency p95	< 300 ms reads, < 600 ms writes	p95 above target for 15 min
Sync success rate	99% of sync calls succeed	< 97% for 30 min
Diagnosis turnaround (corpus path)	p95 < 5 s	p95 > 8 s for 15 min
Payment webhook processing	99.9% within 2 min	Any webhook in DLQ
OTP delivery	98% delivered within 30 s	< 95% for 10 min (switch provider)

12.2 Monitors and workflows

ID	Monitor	Source	Action
OPS-001	Uptime checks on API, console, webhook endpoint from 3 regions	Cloud Monitoring	Page on-call
OPS-002	Error rate and crash-free sessions (target > 99.5%) per app version	Error Reporting, Crashlytics	Halt staged rollout if crash-free < 99%
OPS-003	Cloud SQL CPU > 80%, storage > 80%, connections > 80%, replication lag > 60 s	Cloud Monitoring	Ticket; page if sustained 30 min
OPS-004	Pub/Sub oldest unacked message age > 10 min; any message in DLQ	Cloud Monitoring	Ticket + DLQ replay runbook
OPS-005	Cloud Run Job failures (nightly validation, evaluation, exports, retention purge)	Cloud Monitoring	Ticket; next-morning owner
OPS-006	Officer sync silence: officer on duty with no sync for 2 h	Business metric job	Notify cluster head (possible device issue)
OPS-007	Store with zero sales on a working day, or day not closed by 22:00	Business metric job	Notify store manager and cluster head
OPS-008	Payment reconciliation: Razorpay settlements vs orders mismatch	Daily job	Finance ticket
OPS-009	Diagnosis review queue > 18 h old; LLM fallback rate > 40% in a day	Business metric job	Notify agronomist / product
OPS-010	Validation flags past SLA	Flag service	Escalate to next level
OPS-011	Datastream lag > 30 min; dbt run or test failure	Cloud Monitoring, dbt	Ticket to data owner

ID	Monitor	Source	Action
OPS-012	Cost: daily spend > 130% of 7-day average; LLM and SMS spend by tenant	Billing export + budget alerts	Notify owner + platform lead
OPS-013	Security: SCC high findings, Cloud Armor spikes, OTP send spikes (SMS pumping)	SCC, logs-based metrics	Page on-call
OPS-014	Certificate expiry < 21 days; secret age > 90 days	Scheduled check	Ticket

12.3 Channels and on-call

- Severity 1 (outage, payment failure, data leak): phone + Slack/WhatsApp group, 15-minute response, 24×7 during sowing seasons.
- Severity 2 (degraded, job failure): Slack channel, next business hour.
- Severity 3 (business signals, data quality): daily digest to owners.
- Every Sev 1 gets a blameless postmortem within 5 working days, stored in `docs/postmortems/`.

12.4 Dashboards

- Platform: request rate, errors, latency, instance count, SQL health, queue depth.
- Mobile: active devices, app versions, crash-free rate, sync backlog, offline duration distribution.
- Business ops: officers on duty now, visits today, orders today, payments today, open flags.
- Cost: by service and by tenant.

12.5 Runbooks (write before pilot)

OTP provider outage and failover; payment webhook backlog; Cloud SQL failover and restore; bad mobile release rollback (halt rollout, remote-config kill switch); corpus rollback to previous version; mass location anomaly (OS update breaking background location); data subject erasure request; suspected data breach (DPDP notification).

13. Claude Design prompts

Run five prompts in order in one Claude Design project: the design system first, then one prompt per surface, then the handoff export. Splitting them keeps each result focused and lets you approve the system before screens are built on it. Review each round against the screen inventory before moving on.

13.1 Prompt 1 — design system

```
You are designing the design system for "FarmerApp", a multi-tenant agri-retail platform in India with three surfaces: a field agent Android app (with a store mode for tablets), a farmer Android app, and a web console for owners and managers. Build the design system only in this round; no product screens yet.

Audience and constraints
- Farmers: many have low literacy, use cheap Android phones (360x800 dp, 2 GB RAM), often outdoors in bright sun, with patchy 2G/3G. They read Telugu, Kannada, Hindi, Marathi or English.
- Field officers: young, mobile-first, working all day outdoors on the same kind of phones, often offline.
- Owners and managers: use the web console on laptops (1440x900 and 1024x768) to review data.
- Tone: trustworthy, warm, practical, rural-respectful. Not "startup neon", not corporate grey. Think of a well-run village agri shop: clean, organised, earthy.

Brand and theme tokens (tenant-overridable primary colour and logo)
- Primary "Leaf": #1F6B3A (hover #185630, container #DCEFE2, on-primary #FFFFFF)
- Secondary "Turmeric": #E0A21B (container #FBEFCF, on-secondary #3A2A00) — used for own-brand badges and highlights
- Neutral "Soil": text #2B2520, secondary text #5E554C, muted #8A8076, border #E3DDD3, surface #FFFFFF, background #FAF7F0
- Status: success #2E7D32, warning #B26A00, danger #C62828, info #1565C0, each with a light container tint
- Confidence bands for diagnosis: High = success, Medium = warning, Low/Provisional = neutral grey with dashed border
- Visit status: Verified = success, Partial = warning, Unverified = danger outline
- Themes: (1) Light (default), (2) Sunlight high-contrast for mobile (pure white bg, text #000000, borders 2 px, primary darkened to #134A27, no tinted
```

```
backgrounds), (3) Dark for the web console (bg #151412, surface #1E1C19, text
#F2EEE7, primary #5DBB7E).

Typography
- Font families: Noto Sans for Latin; Noto Sans Telugu, Noto Sans Kannada,
Noto Sans Devanagari for Indic scripts. Numbers use tabular figures.
- Mobile scale (sp): display 28/36, title 22/30, heading 18/26, body 16/24,
label 14/20, caption 12/16. For Indic scripts add ~15% line-height.
- Console scale (px): 28, 22, 18, 15 body, 13 small.
- Weights: 400 and 600 only.
- Show every text style rendered in all five languages with real sample text
(e.g. "Crop doctor", "Pay dues") to prove fit.

Layout
- 4 dp spacing grid (4, 8, 12, 16, 24, 32, 48). Mobile screen padding 16 dp.
Corner radius: 8 (inputs, chips), 12 (cards), 16 (sheets), full (pills).
- Minimum touch target 48 dp; primary actions are full-width bottom buttons
56 dp high.
- Icons: outlined, 24 dp, always paired with a text label on farmer surfaces.
Use simple, culturally familiar pictograms (leaf, bug, drop, sun, rupee,
tractor, sack).
- Elevation: flat surfaces with 1 px borders; one shadow level for bottom
sheets and FAB only.

Components (each with default, pressed, disabled, loading, error, and
Sunlight variants; name them exactly as below because developers will map
them 1:1 to Flutter and React components)
- AppButton (primary, secondary, tertiary, destructive; sizes L and M; with
icon)
- AppTextField (label, helper, error, prefix +91 for phone), OtpInput (6
boxes, auto-advance, resend timer)
- LanguagePicker (large tiles with script name + audio play icon)
- DutyToggle (large switch card showing On duty / Off duty with elapsed time
and a small "location on while on duty" note)
- StatTile (value, label, trend arrow), KpiCard (console)
- FarmerCard (photo or initials, name, village, crop chips, last visit,
balance due badge)
- CropChip (crop icon + name + stage), StageTimeline (sowing → harvest)
- VisitCard (farmer, purpose, time, duration, VisitScoreBadge 0–100 with
status colour)
- ProofChecklist (geofence, photo, farmer confirmed, duration — each
tick/cross with explanation)
- CameraGuideOverlay (leaf frame, tips: "hold steady", "good light", "fill
```

```
the frame")
- DiagnosisResultCard (problem name, ConfidenceBand, symptoms checklist,
"provisional" state)
- TreatmentCard (organic/chemical tag, active ingredient, dose per acre and
per 15 L tank, safety interval, PPE icons)
- ProductCard (image, name, pack, price, OwnBrandBadge, stock at nearest
store, add button)
- CartBar (sticky bottom summary), PaymentMethodSelector (UPI QR, payment
link, cash, credit)
- UpiQrPanel (QR, amount, countdown, "waiting for payment" and "paid" states)
- KhataLedger (balance header, due date, entries list with debit/credit)
- MapPanel (farm polygon, geofence ring, officer track line, visit pins) —
mobile and console versions
- OfflineBanner, SyncStatusPill (synced / pending n / failed), EmptyState
(illustration + one-line help + action), ErrorState, Skeletons
- FlagCard and SeverityBadge (console), DataTable with filters,
DateRangePicker, FilterBar, SideNav, TopBar with tenant switcher
- Toasts, Dialogs, BottomSheets, Stepper (for wizards), Tabs, Segmented
control, Search bar, Rating (1–5 with emoji-free faces or stars)

Deliverables for this round
1. A token page (colour, type, spacing, radius, elevation) for all three
themes.
2. A component library page with every component and state above.
3. A short usage guide per component (when to use, do/don't).
4. Export tokens in W3C Design Tokens JSON format named tokens.json with
semantic names (color.primary, color.surface, text.body, space.4 ...).
Do not design product screens yet.
```

13.2 Prompt 2 — field agent app and store mode

```
Using the approved FarmerApp design system (Light and Sunlight themes),
design the Field Agent Android app ("Mitra") at 360x800 dp, plus Store mode
at 800x1280 dp tablet portrait. Every screen needs default, loading, empty,
error and offline states unless noted. Label each screen with its ID. Use
realistic Indian data (names like Ramesh Reddy, villages like Kondapur, crops
like cotton, chilli, paddy; amounts in ₹ with Indian digit grouping
1,25,000).

Onboarding
A01 Splash with tenant logo. A02 Language picker. A03 Phone login (+91). A04
OTP entry with resend timer. A05 Location and data disclosure: plain
```

language, 4 short points (what is collected, only while on duty, why: travel allowance and visit credit, who can see it), "I agree" button, link to full policy. A06 Permissions explainer then system prompts (location, camera, notifications). A07 Device registered success.

Home and day
A10 Home: DutyToggle at top; today's stats (visits verified, farmers added, collections); "Next up" card from beat plan; quick actions (Add farmer, Start visit, Scout crop, Collect payment); SyncStatusPill; bottom nav (Home, Farmers, Visits, Orders, Me).
A11 Beat plan: ordered list of villages and farmers with reasons (follow-up due, pest alert, credit due), drag to reorder, map toggle.
A12 My day timeline: map with track, stops, visits, gaps; km today; add reason for gap sheet.
A13 Notifications inbox.

Farmers
A20 Farmer list with search, filters (village, crop, due balance), FarmerCards.
A21 Add farmer wizard (Stepper, 5 steps): A21a basics (name, phone, send OTP to farmer, language), A21b farm boundary on map (walk-around mode with live polygon and area in acres, or tap 4 corners; accuracy indicator), A21c crops (crop, variety, sowing date, irrigation, soil), A21d consent (read-aloud button, message consent toggle separate from data consent), A21e review and save (works offline, shows "will sync").
A22 Farmer profile with tabs: Overview (key facts, crops, map), Visits, Diagnoses, Orders, Khata. Primary actions: Start visit, Scout, New order, Collect.
A23 Edit farm / add second farm.

Visits
A30 Start visit: choose farmer/farm, purpose (advisory, scouting, demo, collection, sale, meeting); geofence check result state (inside = green ring, outside = explanation + "request override" with note).
A31 Visit in progress: timer, capture photos (in-app camera only), notes with voice input, crop stage, ProofChecklist live.
A32 Farmer confirmation: choose OTP by SMS or "ask in Kisan app" or show QR; waiting and confirmed states.
A33 Check-out summary with VisitScoreBadge and what was missing.
A34 Visits list (today, week) with status filter.
A35 Group meeting: attendance by scanning farmer QR or OTP, attendee list.
A36 Demo plot: create, stage photos, compare with farmer practice, yield entry.

Scouting
A40 Camera with CameraGuideOverlay. A41 Photo review (retake/use, up to 3). A42 Analysing (queued offline state too). A43 Result: high-confidence variant and provisional variant; TreatmentCards; ProductCards ranked with OwnBrandBadge; "Create order for farmer".

Orders and money
A50 Product search and catalogue with category filters, stock at nearest store. A51 Cart. A52 Payment method. A53 UPI QR waiting/paid/failed. A54 Payment link sent state. A55 Cash receipt capture. A56 Credit sale with limit check and approval-needed state. A57 Order success with share invoice on WhatsApp. A58 Khata for farmer with collect button. A59 Collections list.

Me
A60 My performance (targets vs actual, incentive earned, verified visit trend). A61 Travel allowance: km by day, claims, add expense with receipt photo. A62 My flags (open with "add explanation", resolved with outcome). A63 Profile and settings (language, theme Light/Sunlight, sync now, log out). A64 Sync details (pending items, failed items with retry).

Store mode (tablet, role-gated)
S01 Store home: today's sales, orders queue count, low stock, day-close status. S02 POS: product grid/search + barcode scan, cart panel, farmer lookup by phone. S03 Checkout with payment methods and GST invoice preview. S04 Invoice/receipt share. S05 Order queue (new, packing, ready, handed over). S06 Stock list with batch/expiry, filters. S07 Goods receipt. S08 Stock transfer. S09 Credit approvals. S10 Day close (cash count, variance, deposit slip photo).

Navigation map: provide one flow diagram showing how screens connect. Keep text short, icons labelled, one primary action per screen.

13.3 Prompt 3 — farmer app

```
Using the approved FarmerApp design system, design the Farmer Android app
("Kisan") at 360x800 dp in Light and Sunlight themes. Design for low
literacy: big icons with labels, photos, audio play buttons next to key
instructions, no typing except phone and OTP, maximum 2 decisions per screen.
Show every screen in Telugu and English at minimum (Hindi for the home and
result screens too). Include loading, empty, error and offline states.

F01 Language picker with audio. F02 Phone login. F03 OTP. F04 Consent: two
separate cards ("use my data to give crop advice" required, "send me offers
and tips" optional), audio read-aloud, simple words. F05 Welcome: if an
officer already registered them, show "Your farm details from Ramesh (field
officer)" to confirm; else quick setup (crop + village).

F10 Home: greeting, weather strip for farm location with spray advice (good /
avoid), a very large "Check my crop" button (camera icon), My crops cards
with stage and next task, alerts banner for pest outbreaks in district, dues
card if balance > 0, bottom nav (Home, Crop doctor, Orders, Me).

F20 Crop doctor intro with 3 picture tips. F21 Camera with guide overlay. F22
Photo check (blur warning with retake). F23 Choose crop if not detected (big
crop tiles). F24 Analysing (with offline "we'll check when you're online").
F25 Result high confidence: problem photo, name in local language, "Check
these signs" symptom ticks, treatments tabs Organic / Chemical, dose per acre
and per 15 L tank shown visually (tank icon), safety days and PPE pictograms,
products available at "FCC Kondapur store, 3 km" with price and own-brand
badge, buttons Order / Call my officer. F26 Result provisional: "Our expert
will confirm within 24 hours", no buy button, call officer button. F27 Result
updated notification state. F28 Not a plant / unsupported crop.

F30 Product detail. F31 Cart and pickup or delivery choice. F32 Pay: UPI
(opens UPI apps) or "Pay at store" or credit request. F33 Order tracking
(placed, packed, ready, collected). F34 My orders.

F40 My crops and farms. F41 Add crop (tiles + sowing date picker with
calendar). F42 Crop calendar: timeline of tasks with reminders on/off. F43
Weather 7-day detail. F44 Alerts list.

F50 Khata: balance, due date, history, Pay now. F51 Payment success receipt.

F60 Visit confirmation prompt ("Did Ramesh visit your farm today?" Yes/No)
and F61 rating (5 faces + quick reasons). F62 My field officer card (photo,
```

```
name, call, WhatsApp).

F70 Me: language, theme, notifications, consent settings (withdraw
marketing), help (call store), delete my account request, about.

Provide a navigation flow diagram. Keep the whole app feeling calm and
trustworthy; avoid dense tables anywhere.
```

13.4 Prompt 4 — web console

```
Using the approved FarmerApp design system (Light and Dark themes), design the web console at 1440x900 with a responsive check at 1024x768. Layout: left SideNav (collapsible), TopBar with tenant name, global search (farmer phone/name, officer, order id), date range, notifications, profile. Every data view has FilterBar (cluster, store, officer, crop, date), export button, and empty/loading/error states. Use realistic data for 24 stores in Telangana, Karnataka and Maharashtra.

C01 Login (phone + OTP; SSO option). C10 Owner home: KPI row (revenue, own-brand share, credit outstanding and overdue, verified visits, active farmers, open flags) with targets and trend; revenue and own-brand share line chart; store map coloured by performance; top/bottom 5 officers; open high-severity flags; crop health alerts.
C20 Field team list (scorecard columns: verified visits/day, unique farmers, confirmation rate, time on farm, km/day, conversion, collections, flags) with sparkline trends. C21 Officer profile. C22 Day replay: map with track, stops, visit pins coloured by status, gaps; right panel timeline with visit details and ProofChecklist; playback slider.
C30 Farmers directory. C31 Farmer 360: profile, farms map, crops, visit history, diagnoses with photos, orders, khata, consent status, app usage. C32 Segments builder (crop, district, stage, spend, last visit, consent) with live count.
C40 Stores list. C41 Store detail (sales by category and own-brand, margin, stock, credit ageing, day-close variance).
C50 Product master. C51 Label claims editor (SKU x crop x pest x dose x PHI) with validation. C52 Price lists.
C60 Agronomist review queue (image gallery, model candidates with scores, pick/correct label, add to corpus, notify farmer). C61 Corpus manager (problems, images grid, status workflow draft/reviewed/approved/retired, version history). C62 Diagnosis quality dashboard.
C70 Validation inbox (FlagCards grouped by severity and rule, SLA timers). C71 Flag detail: evidence bundle (map, records, photos), AI summary box clearly labelled "Automated summary — verify evidence", actions (valid, corrected, escalate, action taken) with notes, officer explanation thread.
C80 Campaigns list. C81 Campaign builder (segment, channel SMS/WhatsApp/push, message in multiple languages, schedule, preview on phone). C82 Campaign results.
C90 Targets and incentives (per officer/store, rule editor, payout preview). C91 Travel allowance approvals.
C100 Reports and scheduled exports. C101 Users and roles, device bindings, force logout. C102 Settings (geofence radius, min visit time, TA rate, credit
```

```
limits, languages, branding with live preview of both apps). C103 Audit log
viewer.
C110 Platform admin (P2): tenants list, create tenant, plans, usage and
billing.

Charts: line for trends, bars for comparisons, maps for geography; no pie
charts except own-brand share donut on C10. Provide a navigation map.
```

13.5 Prompt 5 — handoff export

```
Prepare the developer handoff for Claude Code.
1. Export tokens.json (W3C format) for Light, Sunlight and Dark themes.
2. For each screen ID, export a PNG at 1x and 2x and a spec file
screens/<ID>.md listing: purpose, components used (by exact component name),
data fields shown with example values, actions and where they navigate,
states covered, and copy text keys (e.g. home.checkCrop.title) with English
text.
3. components/<Name>.md for every component: props, variants, states,
accessibility notes, and the token names it uses.
4. A flows.md with the navigation maps for all three surfaces.
5. A copy.csv with every string key and the English text so translators can
add Telugu, Kannada, Hindi and Marathi.
Name everything consistently with the screen IDs A.., S.., F.., C.. used in
the prompts.
```

13.6 Screen inventory checklist

Surface	Screen IDs	Count
Field agent app	A01–A07, A10–A13, A20–A23 (incl. A21a–e), A30–A36, A40–A43, A50–A59, A60–A64	45
Store mode	S01–S10	10
Farmer app	F01–F05, F10, F20–F28, F30–F34, F40–F44, F50–F51, F60–F62, F70	31
Web console	C01, C10, C20–C22, C30–C32, C40–C41, C50–C52, C60–C62, C70–C71, C80–C82, C90–C91, C100–C103, C110	28

Design is complete when every ID exists in each required theme with its states, and the handoff files in 13.5 are committed to `design/`.

14. Claude Code build plan and loop prompts

Build in nine milestones, each small enough for a looping Claude Code session to finish against a checkable definition of done. The pilot (M0–M5) targets field use in 5 stores in December 2026; the farmer app follows for Rabi scouting, and full rollout lands before Kharif 2027.

14.1 Milestones

#	Milestone	Requirement IDs	Target	Definition of done
M0	Foundations: repo, CLAUDE.md, docker compose, Terraform bootstrap + dev env, CI skeletons, auth + tenants + users + RLS, OpenAPI + generated clients, design system in Flutter and React	FA-001–004, FA-006, SEC-001–006, SEC-010–013	Oct 9, 2026	Login works end to end on dev; RLS tests green; all CI pipelines green
M1	Farmers and farms: registration wizard, map boundary, crops, consent, offline sync engine	FA-012–013, FA-025, FM-002 (data model), NF-002–004	Oct 30, 2026	72-hour offline test passes; duplicate checks work
M2	Duty, location and visits: foreground tracking, geofence, visit flow, farmer OTP confirmation, scoring	FA-010–011, FA-014–017, FA-028, VV-001–005, VV-007, VV-009–010	Nov 20, 2026	Battery budget met on reference device; visit score correct on fixtures
M3	Catalogue, store mode, orders, payments, khata	FA-020–022, ST-001–003, ST-005–007, WC-005, SEC-023	Dec 4, 2026	Razorpay test-mode UPI flow and webhook reconciliation pass; GST invoice validated

#	Milestone	Requirement IDs	Target	Definition of done
M4	Console v1 and validation agents v1: owner home, team scorecard, day replay, farmer 360, flag inbox, P0 rules	WC-001–004, WC-010–012, AN-001–003, AN-011, VA-001–003, VA-007, VA-F01–F03, VA-F06, VA-F08, VA-E01–E05, VA-E08	Dec 11, 2026	Rule fixtures green; owner can see yesterday's field activity
M5	Pilot hardening: monitoring, alerts, runbooks, pen test fixes, staging → prod	OPS-001–010, SEC-014–019, SEC-024–026, NF-005–006	Dec 18, 2026	Pilot go-live checklist signed; 5 stores live
M6	Diagnosis engine + farmer app v1	DX-001–010, FM-001–006, FM-010–011, FM-014–015, FA-019, WC-006–007, A40–A43, F-screens	Feb 26, 2027	Corpus answers ≥ 70% of pilot test set; farmer app in Play closed testing
M7	Analytics and growth: BigQuery + dbt, campaigns, calendar, weather, alerts, incentives, TA, remaining validation rules	AN-004–015, WC-008–009, WC-013, FM-007–009, FA-018, FA-023–024, FA-026–027, DX-011–013, VA-004–006, remaining VA rules	May 14, 2027	All 24 stores live; weekly owner email running
M8	SaaS readiness: branding, tenant admin, billing, iOS farmer app	FA-008, WC-014, NF-007, AN-016	FY28 Q1	Second tenant onboarded in under 30 minutes

14.2 CLAUDE.md (commit at repo root)

```
# FarmerApp — instructions for Claude Code

## Source of truth
- docs/SPEC.md is the specification. Requirement IDs (FA-, FM-, ST-, WC-,
DX-, VV-, AN-, VA-, NF-, SEC-, OPS-) define scope.
- design/ holds the approved design: tokens.json, screens/<ID>.md,
components/<Name>.md, copy.csv. Match them exactly.
```

```
- docs/progress.md tracks status per requirement ID: todo | in-progress |
done | blocked (with reason).

## Stack (do not change without an ADR in docs/adr/)
API/worker/jobs: Python 3.12, FastAPI, SQLAlchemy 2, Alembic, Pydantic v2,
pytest.
Console: Next.js (App Router), TypeScript strict, Tailwind, shadcn/ui,
TanStack Query.
Mobile: Flutter, Riverpod, Drift, WorkManager; shared code in
apps/mobile/packages/core.
Infra: Terraform in infra/. Analytics: dbt in dbt/.

## Rules
- Work one requirement ID at a time. Write or update tests first;
implementation second.
- Every table has tenant_id and RLS; every endpoint has role checks and an
authz test for every role.
- Money is integer paise. Timestamps UTC in DB. Strings go in i18n files,
never hard-coded.
- Never recommend a SKU without a matching label_claim (DX-007).
- Location is collected only on duty (FA-010). Never add hidden tracking.
- No secrets in code, fixtures or logs. Use Secret Manager references and
.env.example.
- Migrations must be backward compatible.
- Run `make check` before declaring anything done; it must pass.
- Update docs/progress.md and the OpenAPI spec in the same change.

## Never do (human-only)
- terraform apply to stg or prd; deploy to prd; publish to Play Console.
- Create or rotate real secrets; call real payment or SMS providers outside
test mode.
- Delete data in any shared environment.

## Commands
make up | make seed | make check | make test-api | make test-console | make
test-mobile | make plan-dev
```

14.3 Loop prompt (use for every milestone)

Paste this as the repeating prompt, replacing `<M#>`. Each iteration does one requirement end to end and stops cleanly, so the loop can run until the milestone is done.

```
You are building FarmerApp. Read CLAUDE.md, docs/SPEC.md section 14.1 row
<M#>, and docs/progress.md.

1. If every requirement ID in <M#> is "done" and `make check` passes and the
<M#> definition of done is met, write a short milestone report to
docs/milestones/<M#>.md and reply exactly "MILESTONE <M#> COMPLETE". Stop.
2. Otherwise pick the first <M#> requirement that is "todo" or "in-progress"
(skip "blocked"). Mark it in-progress.
3. Read its acceptance criteria in SPEC.md and any linked design screens in
design/screens/.
4. Write failing tests that encode the acceptance criteria (unit +
API/integration; widget or Playwright tests for UI).
5. Implement the smallest change that makes them pass, following CLAUDE.md
rules and the design tokens/components.
6. Run `make check`. Fix until green. Do not skip or delete tests to make
them pass.
7. Update OpenAPI, generated clients, i18n keys, and docs/progress.md (mark
done, note files changed).
8. Commit on a branch named feat/<ID>-<slug> with message "feat(<ID>):
<summary>" and open a PR description in docs/prs/<ID>.md.
9. If blocked by a missing decision, credential, or ambiguity, mark the
requirement "blocked" with the exact question in docs/progress.md and move to
the next one.
Report in 5 lines: ID worked on, tests added, result of make check, what is
next, any blockers.
```

14.4 Milestone kickoff prompts

Run once at the start of a milestone, before the loop.

- **M0:** "Scaffold the monorepo exactly as SPEC 10.1. Create docker-compose with Postgres 16 + PostGIS + pgvector, Pub/Sub emulator, fake GCS and mock SMS/payment servers. Create Makefile targets from CLAUDE.md. Implement Terraform modules in SPEC 9.1 for dev only and the bootstrap. Create CI workflows from SPEC 10.3 (deploy to dev only). Generate docs/progress.md listing every requirement ID in SPEC with status todo. Then implement the design system from design/tokens.json and design/components in both Flutter (packages/core) and React (console) with golden/visual tests."
- **M1–M3:** "Create the Alembic models for the entities in SPEC 8.2 needed by this milestone, with RLS policies and seed data for a demo tenant (5 stores, 5 officers, 200 farmers in Telangana). Then start the loop."

- **M4:** "Implement the rules engine: load rules/*.yaml, run inline/event/batch modes per SPEC 6.1, store flags per VA-002. Add positive and negative fixtures for each P0 rule. Then start the loop."
- **M6:** "Build the diagnosis pipeline per SPEC 3 with a pluggable encoder and LLM client, pgvector HNSW index, evaluation script over ml/eval/testset. Use a small public seed set marked source=public. Then start the loop."
- **M7:** "Enable Datastream to BigQuery in dev via Terraform, scaffold dbt staging and marts for the KPIs in SPEC 5.2 with tests, and wire console dashboards to marts. Then start the loop."

14.5 Global definition of done

- [ ] Acceptance criteria covered by automated tests; `make check` and CI green.
- [ ] Matches the design screen spec (screenshot or golden test attached to the PR).
- [ ] All strings externalised; English plus at least one Indic language verified on device.
- [ ] Role and tenant authorisation tests added.
- [ ] Logs, metrics and alerts added where the requirement has an OPS or SLO link.
- [ ] OpenAPI, progress.md and any runbook updated.
- [ ] Reviewed and merged by a human; deployed to dev and smoke-tested.

14.6 Human gates

These stay with you or a named reviewer, never with the loop: production Terraform applies and deploys, Play Console releases, secrets and provider accounts (Razorpay, SMS DLT registration, LLM keys), agronomist approval of corpus entries and label claims, the employee location policy and farmer consent text, and acceptance of each milestone.